



















































































































































































































































































































































































Ahh yes 😊 — I made that part sound more mysterious than it is. Let's strip it down.

## What is an auth library/provider?

When you build authentication **yourself**, you have to write a lot of infrastructure:

```
Login form
  ↓
Verify password
  ↓
Create session
  ↓
Store session
  ↓
Set cookie
  ↓
Read cookie
  ↓
Find current user
  ↓
Handle logout
  ↓
Protect routes
  ↓
```



```
Handle OAuth
  ↓
Refresh/expire sessions
  ↓
...
```

You *can* build all of this yourself — and that's what we've been learning.

But in a real production application, you often don't want to reinvent all of this authentication machinery.

That's where an **authentication library** comes in.

---

## Think of it like this

Suppose you want login with:

```
Email + Password
Google
GitHub
Apple
Microsoft
```



If you build everything yourself, you'd need to implement and maintain all the authentication flows.

Instead, you can use an auth library.

For example, in the Next.js ecosystem, libraries such as **Auth.js** provide authentication infrastructure that your application can integrate.

The library handles a lot of the repetitive authentication plumbing.

Your application still decides things like:

```
Who is the user?
What roles do they have?
What are they allowed to access?
```



---

## What does "provider" mean?

This is where the terminology gets confusing. 🤔

An **authentication provider** generally means the system that actually authenticates the user.

For example:

## Google

```
graph TD
    A[Your app] --> B["Login with Google"]
    B --> C[Google]
    C --> D[Google authenticates user]
    D --> E[Google tells your app]
    E --> F["Yes, this user successfully authenticated"]
    F --> G[Your app creates/gets its session]
```

Google is acting as an **identity provider (IdP)**.

Other examples:

```
graph TD
    A[Google]
    B[GitHub]
    C[Microsoft]
    D[Apple]
    E[Okta]
    F[Auth0]
```

They can authenticate users on your application's behalf.

---

## Example: Login with Google

Without Google:

```
graph TD
    A[Browser] --> B[Your Login Page]
    B --> C[Your Next.js Server]
    C --> D[Your DB]
    D --> E[Verify password]
```



↓  
Create session

With Google:

Browser  
↓  
Your App  
↓  
Google  
↓  
User logs into Google  
↓  
Google → Your App  
↓  
Your App knows who the user is

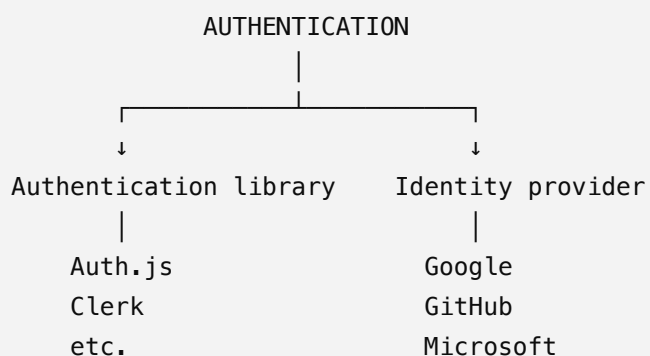
Your application doesn't need to collect the user's Google password.

That's a **provider**.

## But what's an "auth library" then?

This is a different layer.

Think:

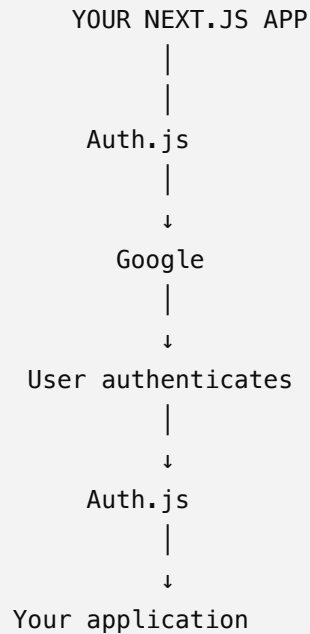


The library provides the **authentication machinery/integration**.

The provider may be the service that actually authenticates the person.

## Example with Auth.js + Google

Conceptually:



The library handles things like the OAuth flow, callbacks, session handling/integration, etc., depending on how you've configured it.

## And this connects to the Client Component thing

Earlier I said something like:

```
"use client";

const session = useSession();
```

This is **not magic built into React**.

A library can provide a client-side API/hook that lets your UI access session information.

For example, conceptually:

```
"use client";

function Navbar() {
  const session = useSession();

  if (session) {
    return <p>Welcome {session.user.name}</p>;
  }
}
```

```
return <a href="/login">Login</a>;  
}
```

The library might maintain/provide session information to the client.

But remember the crucial distinction:

## Client

```
"Should I show the Logout button?"  
"Should I show the Profile menu?"  
"Should I show the Login button?"
```



## Server

```
"Is this user actually authenticated?"  
"Is this user allowed to delete this account?"  
"Is this user an ADMIN?"
```



The **server** remains the security boundary.

---

## You don't actually need an auth library

This is important.

You could absolutely build:

```
Next.js  
  ↓  
Your login Server Action  
  ↓  
Argon2  
  ↓  
Your DB  
  ↓  
Your session table  
  ↓  
HttpOnly cookie
```



That's perfectly valid.

We're actually learning this architecture first because I want you to understand **what authentication is doing underneath the library**.

Then when you encounter:

```
useSession()
```

or:

```
Auth.js  
Clerk  
Auth0
```



you won't think:

"Oh, this library somehow magically knows who the user is." 😂

You'll understand:

**"This is an abstraction over authentication/session machinery that ultimately still has to establish and verify authenticated state on the server."**

## One more distinction: Auth library vs provider

| Term              | Meaning                                                              | Example                   |
|-------------------|----------------------------------------------------------------------|---------------------------|
| Auth library      | Software your application uses to implement/integrate authentication | Auth.js                   |
| Identity provider | Service that authenticates the user                                  | Google, Microsoft, GitHub |
| Your server       | Enforces access to your application's resources                      | Next.js                   |
| Database          | Stores your application's user/session data as configured            | PostgreSQL, MySQL, etc.   |

So if you say:

**"We're using Google authentication."**

You're talking about the **provider**.

If you say:

**"We're using Auth.js."**

You're talking about the **authentication library**.

And they can be used **together**.

## The simplest mental model 🧠



So **auth library**  $\neq$  **provider**  $\neq$  **database**  $\neq$  **authorization**. They're separate pieces that can work together. 🔥



if we hava auth library why we need provider again and explain under the hood of auth.js and provider with code example i coukdnt understnad

Yes — this is the exact point where “auth library” and “provider” sound like two things doing the same job. They aren't. 😊

The easiest way to understand it is:

**Auth.js is the machinery. Google is one possible identity system that the machinery talks to.**

Let's go under the hood with an actual flow.

---

## 1. First: why do we need a provider at all?

Suppose you use Auth.js.

You write:

`</> TypeScript`



```
providers: [  
  Google(...)  
]
```

You might think:

“Wait, if Auth.js handles authentication, why does it need Google?”

Because **Auth.js doesn't know the user's Google password.**

Imagine Gopi clicks:

Continue with Google



Who is going to answer:

“Is this actually Gopi? Did she successfully authenticate?”

Auth.js cannot answer that by itself.

**Google can.**

So:

```
Auth.js  
  ↓  
"Google, please authenticate this user"  
  ↓  
Google  
  ↓  
"Yes, this user successfully authenticated"  
  ↓  
Auth.js
```



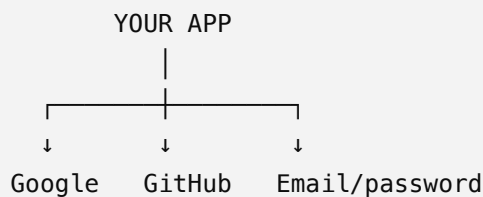
That's the fundamental reason.

---

## 2. Think of Auth.js as a receptionist 🏢

Imagine your application is a building.

You have three possible ways someone can enter:



Auth.js is the **reception/security system** that knows how to handle these different login methods.

Google/GitHub are **identity providers** that can authenticate the person.

## 3. Let's use Google as the provider

Your Next.js application might have:

❏ TypeScript

// auth.ts

```
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID,
      clientSecret: process.env.AUTH_GOOGLE_SECRET,
    }),
  ],
});
```

This configuration basically tells Auth.js:

“Hey Auth.js, Google is one of the ways users can authenticate with my application.”

Auth.js officially supports provider integrations such as GitHub and many others.



## 4. What happens when user clicks "Sign in with Google"?

Suppose your UI has:

```
</> TypeScript
```

```
<button>  
  Sign in with Google  
</button>
```

The flow is approximately:

```
Browser  
  |  
  | "I want to login with Google"  
  ↓  
Next.js / Auth.js  
  |  
  | redirect user  
  ↓  
Google  
  |  
  | Google login page  
  ↓  
User enters Google credentials  
  |  
  ↓  
Google authenticates user  
  |  
  | authorization code  
  ↓  
Next.js / Auth.js  
  |  
  | exchange code with Google  
  ↓  
Google  
  |  
  | user identity/tokens  
  ↓  
Auth.js  
  |  
  | create application session  
  ↓  
Browser
```

Now let's slow down because **this is the important part**.

## 5. Step 1 — Your application sends the user to Google

User clicks:



Continue with Google



Your app essentially starts an OAuth flow.

Conceptually:

Browser



GET /api/auth/signin/google



Auth.js



Auth.js knows:

`</> TypeScript`

```
Google({  
  clientId: "...",  
  clientSecret: "..."  
})
```



So it knows how to construct the Google authorization request.

Then the browser gets redirected to something conceptually like:

`https://accounts.google.com/...`



The exact URL/query parameters are more complicated, but conceptually it's:

Google:

"Hey Google, authenticate this user  
for my application."



---

## 6. Step 2 — Google authenticates the user

Now we're on Google's website:

GOOGLE

|          |
|----------|
| Sign in  |
| Email    |
| Password |



[Continue]

Notice something VERY important:

**Your Next.js application never receives the Google password.**

The user enters it into **Google's website**.

So:

Google password



Google



not:

Google password



Your Next.js server ❌



That's one huge advantage of OAuth providers.

---

## 7. Step 3 — Google says "authentication succeeded"

After the user successfully authenticates, Google redirects the browser back to your application.

Conceptually:

Google



<https://yourapp.com/api/auth/callback/google>



And Google gives your application an **authorization code**.

Something conceptually like:

code=ABC123



Important:

**The authorization code is not the user's password.**

## 8. Step 4 — Auth.js receives the callback

Your Next.js application receives:

```
GET /api/auth/callback/google?code=ABC123
```



Auth.js handles this callback.

This is one reason the library is useful.

You don't have to manually implement all of the OAuth protocol handling.

Auth.js knows:

```
How Google OAuth works
How to validate the callback
How to exchange the authorization code
How to obtain user identity information
How to create/manage the application session
```



---

## 9. Step 5 — Auth.js talks to Google's server

This part is REALLY important.

The browser isn't simply saying:

```
"Google says I'm Gopi, trust me."
```



No.

Your server/Auth.js communicates with Google using the configured OAuth credentials and exchanges the authorization code for the appropriate tokens/identity information.

Conceptually:

```
Next.js server
  |
  | authorization code
  ↓
Google
  |
  | tokens / identity information
  ↓
Next.js server
```



Now Auth.js can establish:

```
This Google account
      ↓
belongs to
      ↓
this application user
```



## 10. Step 6 — Auth.js creates the application's session

This is the part that connects back to everything we've already learned.

After successful authentication:

```
Google authentication
      ↓
Auth.js
      ↓
Application session
      ↓
Browser cookie
```



The browser might end up with an Auth.js session cookie.

Then later:

```
Browser
  |
  | GET /dashboard
  | Cookie: <session>
  ↓
Next.js
  ↓
Auth.js
  ↓
Current user
```



So after the initial Google authentication, **Google doesn't need to authenticate you again on every request to your app.**

Your application's session handles that.

## 11. This is the distinction you were missing

There are actually **two different authentication relationships** here.

## Relationship #1

User ↔ Google



Google establishes:

"This person successfully authenticated to Google."

## Relationship #2

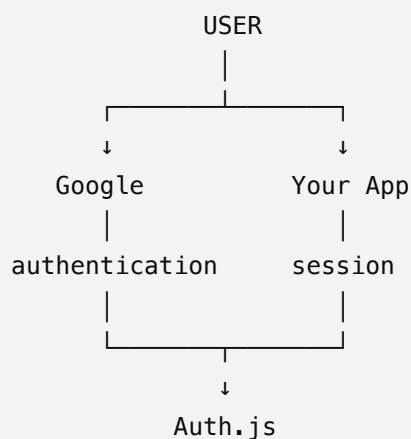
User ↔ Your Application



Auth.js establishes:

"This authenticated person has a session in our application."

So:



## 12. Now you'll understand why Auth.js needs providers

Suppose tomorrow you want:

Login with Google  
Login with GitHub  
Login with Microsoft



You can configure:

</> TypeScript



```
export const { handlers, auth } = NextAuth({
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID,
      clientSecret: process.env.AUTH_GOOGLE_SECRET,
    }),

    GitHub({
      clientId: process.env.AUTH_GITHUB_ID,
      clientSecret: process.env.AUTH_GITHUB_SECRET,
    }),
  ],
});
```

Now Auth.js knows:

Google → use Google's OAuth protocol/configuration



GitHub → use GitHub's OAuth protocol/configuration

Auth.js is the **common authentication framework**.

Each provider is a **different identity system**.

---

## 13. What if you DON'T want Google?

You don't have to use an external provider.

You can have:

Email  
Password



and authenticate against **your own database**.

Conceptually:

```
Browser
  ↓
Next.js
  ↓
Auth.js / your auth logic
  ↓
Your DB
  ↓
Verify password hash
  ↓
```



Create session

↓

Cookie

Here, **your application is responsible for authenticating the password.**

Google isn't involved.

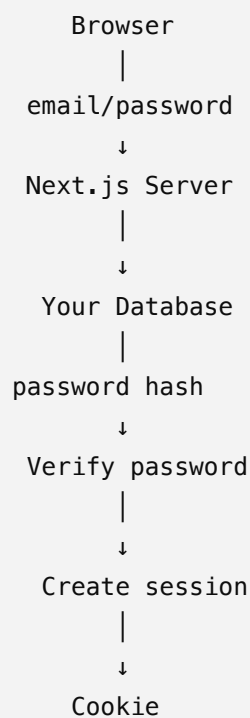
So the provider is not mandatory in the sense of "you must use Google."

An auth library can support different authentication mechanisms, including credentials and external OAuth providers depending on its configuration.

## 14. And here's the BIG picture

Compare the two systems.

### Your own email/password authentication



```
graph TD; Browser --> email_password[\"email/password\"]; email_password --> Nextjs_Server[\"Next.js Server\"]; Nextjs_Server --> Your_Database[\"Your Database\"]; Your_Database --> password_hash[\"password hash\"]; password_hash --> Verify_password[\"Verify password\"]; Verify_password --> Create_session[\"Create session\"]; Create_session --> Cookie;
```

Browser

↓

email/password

↓

Next.js Server

↓

Your Database

↓

password hash

↓

Verify password

↓

Create session

↓

Cookie



### Google authentication



```
graph TD; Browser --> login_google[\"\"Login with Google\""]; login_google --> Nextjs_Authjs[\"Next.js/Auth.js\"]; Nextjs_Authjs --> Nextjs_Authjs;
```

Browser

↓

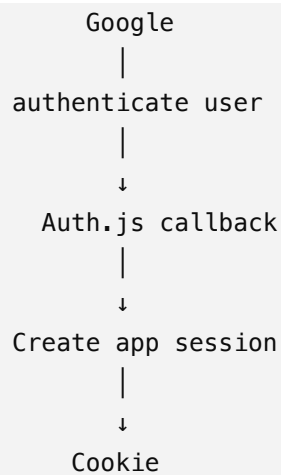
"Login with Google"

↓

Next.js/Auth.js

↓





## 15. So what exactly does Auth.js save us from writing?

This is the real value.

Without a library, you'd need to understand and implement things like:



```
OAuth redirect
  ↓
state parameter
  ↓
authorization code
  ↓
callback handling
  ↓
code/token exchange
  ↓
provider-specific APIs
  ↓
identity mapping
  ↓
session management
  ↓
cookies
  ↓
expiration
  ↓
sign out
  ↓
CSRF protections
  ↓
etc.
```

Auth.js packages much of that authentication infrastructure into a reusable framework. Its official Next.js setup exposes handlers and an `auth()` API, and its provider system lets you configure integrations such as GitHub.  Auth.js +1



## 16. One thing I DON'T want you to misunderstand

Don't think:

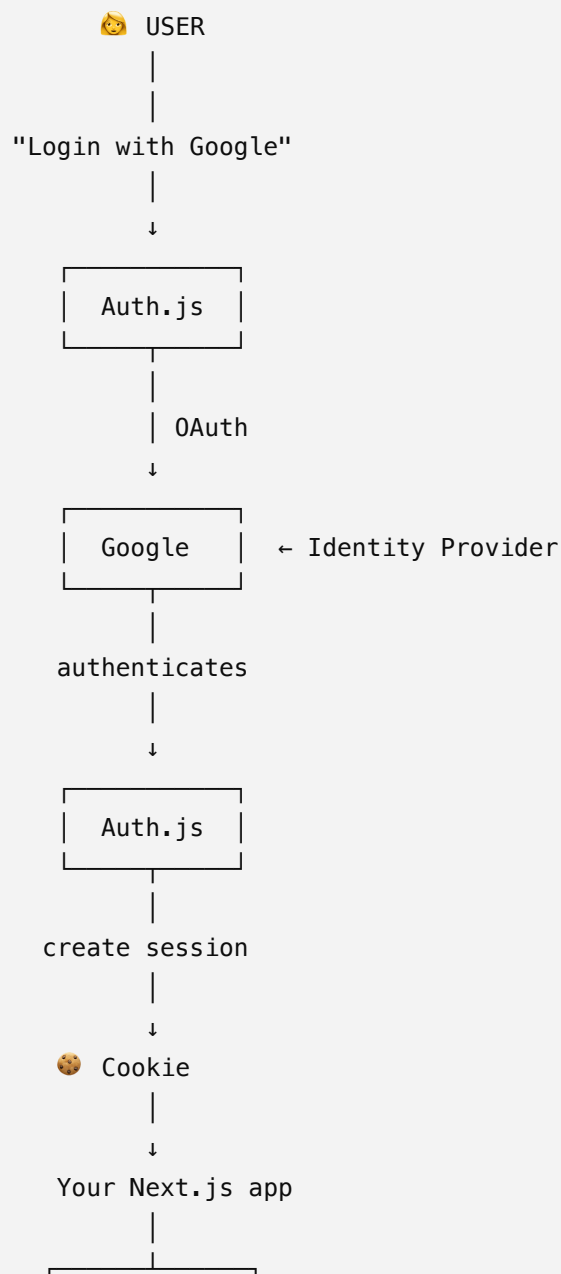
"Auth.js authenticates the user, and Google also authenticates the user, so why do both?"

Instead think:

"Google is one identity provider that can authenticate the user. Auth.js is the machinery in my application that integrates that authentication method and then manages the user's application session."

That's the clean mental model. 🧠

In one diagram:



↓                      ↓  
Auth check      Authorization

And **this is why we're learning authentication manually first**: once you understand the cookie/session flow we built earlier, Auth.js becomes an abstraction you can mentally unpack instead of a magical black box. 😊



























































































































































































































































































































Awesome 😊 Let's move to the next important Auth.js concept: how the session is actually used to protect pages and APIs.

## Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
  ↓
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
```



↓  
Session  
↓  
Cookie

Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

## 1. Protecting a Server Component

Suppose we have:

```
app/  
├── login/  
│   └── page.tsx  
└── dashboard/  
    └── page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
</> TypeScript  
  
// app/dashboard/page.tsx  
  
import { auth } from "@auth";  
import { redirect } from "next/navigation";  
  
export default async function DashboardPage() {  
  const session = await auth();  
  
  if (!session) {  
    redirect("/login");  
  }  
  
  return (  
    <div>  
      <h1>Welcome {session.user?.name}</h1>  
      <p>{session.user?.email}</p>  
    </div>  
  );  
}
```

Now let's understand **exactly what happens**.

---

## 2. User requests `/dashboard`

Browser:

```
</> http  
  
GET /dashboard  
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript  
  
const session = await auth();
```



runs.

---

## 3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript  
  
// auth.ts  
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut  
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:

```
Browser  
|
```





```
    | Cookie: session-token=ABC123
    ▼
Next.js
  |
  ▼
Dashboard Server Component
  |
  | auth()
  ▼
Auth.js
  |
  ▼
Session verification / lookup
  |
  ▼
User session
```

If the session is valid:

</> TypeScript



```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

---

## 4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

</> TypeScript



```
const session = await auth();
```

returns something equivalent to:

</> TypeScript



```
null
```

So:

```
</> TypeScript

if (!session) {
  redirect("/login");
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http

307 Temporary Redirect
Location: /login
```



The browser then requests:

```
</> http

GET /login
```



So:

```
User → /dashboard
      ↓
      auth()
      ↓
      no session
      ↓
      redirect("/login")
      ↓
      /login
```



---

## 5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript

if (!session)
```



That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

</> TypeScript



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

---

## 6. But there's an important security rule 🚨

Never rely only on hiding UI.

For example:

</> TypeScript



```
{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

---

## 7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```

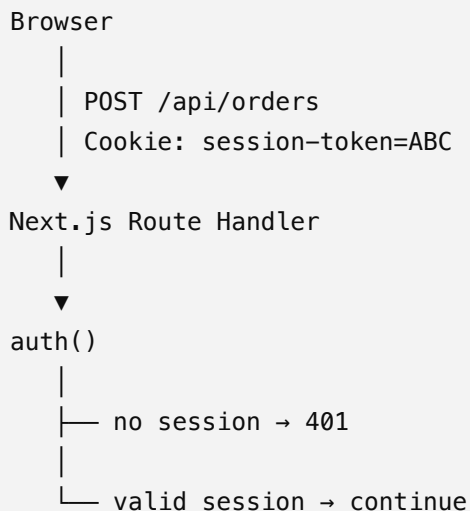


We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```

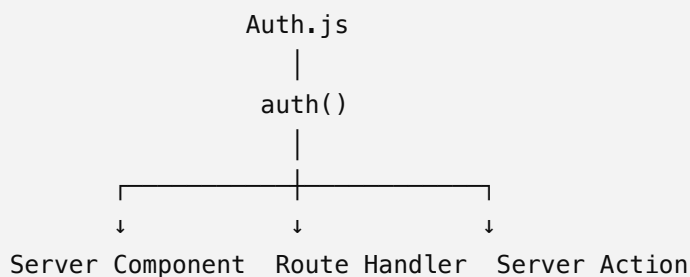


Now:



## 8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

### Server Component

`</>` TypeScript

```
const session = await auth();
```

### Route Handler

`</>` TypeScript

```
const session = await auth();
```

### Server Action

`</>` TypeScript

```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

**Your server-side code asks Auth.js to resolve the current authenticated user/session.**

## 9. Server Action example

Suppose:

```
</> TypeScript

"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

## 10. Interview answer 🎯

If an interviewer asks:

**"How would you protect a page in Next.js using Auth.js?"**

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and

authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

## One thing I want you to notice

We now have **three different places** where Auth.js can participate:



```
1. LOGIN
  signIn()
    ↓
  OAuth
    ↓
  callback
    ↓
  session created

2. READING SESSION
  auth()
    ↓
  "Who is currently logged in?"

3. PROTECTING ACTIONS
  auth()
    ↓
  authenticate
    ↓
  authorize
    ↓
  perform operation
```

That distinction is **very interview-important**.

## Next after this

The natural next piece is **Auth.js callbacks** — specifically:

&lt;/&gt; TypeScript



```
callbacks: {  
  async jwt() {},  
  async session() {},  
  async signIn() {},  
  async redirect() {}  
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

### 🔒 Remaining Auth.js / Auth & Authorization lessons

| #    | Topic                   | What you'll learn                                                                               |
|------|-------------------------|-------------------------------------------------------------------------------------------------|
| 7.6  | 🛡️ Route Protection     | <code>auth()</code> , protecting pages, APIs, Server Actions                                    |
| 7.7  | 🔄 Auth.js Callbacks     | <code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks |
| 7.8  | 👤 Session Customization | Put <code>user.id</code> , <code>role</code> , etc. into the session                            |
| 7.9  | 🔑 Authorization / RBAC  | Roles, permissions, admin/user access                                                           |
| 7.10 | 🍪 Session Strategies    | Database sessions vs JWT sessions                                                               |
| 7.11 | 🔒 CSRF + SameSite       | Why state-changing requests need protection                                                     |



| #    | Topic                                                                                                        | What you'll learn                                               |
|------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 7.12 |  Middleware / Proxy Auth    | Protecting groups of routes before rendering                    |
| 7.13 |  Logout & Session Expiry    | Sign-out, expiration, invalid/expired sessions                  |
| 7.14 |  Complete Auth Architecture | Put Browser → Next.js → Auth.js → OAuth → DB → Session together |
| 7.15 |  Auth Interview Questions   | Real interview scenarios + architecture questions               |

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization section complete**.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:



## AUTHENTICATION

---

- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

## AUTHORIZATION

---

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

## SECURITY

---

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD
    Browser --> Login
    Login --> Authjs[Auth.js]
    Authjs --> Google[Google OAuth/OIDC]
    Google --> Callback
    Callback --> Token[Google token exchange]
    Token --> User[User + Account]
    User --> Adapter
    Adapter --> Database
    Database --> Session
    Session --> Cookie
    Cookie --> Request[Protected request]
    Request --> Auth[auth()]
    Auth --> Authn[Authentication]
    Authn --> Authz[Authorization]
    Authz --> Resource[Protected resource]
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

**So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE.** 🗝️✅

📄 👍 📤 🔄 ...

ChatGPT can make mistakes. Check important info.